

Provenance-Native Storage

SUBProvenance-Native Storage

p-rick research program

p-rick — an independent research program on missing software: Causal provenance at the personal storage layer.

Provenance-Native Storage

SUBProvenance-Native Storage

p-rick research program

Working Paper P-005 · v1.0 · October 7, 2026 · CC BY 4.0

Abstract — Every file on a personal computer is an amnesiac. It knows its size and its modification time; it has forgotten who created it, from what inputs, through which transformations, over which network origin, and with which degree of trust. Twenty years ago the research community proved that operating systems *can* record this causal history automatically — the Provenance-Aware Storage System (PASS, FAST 2006) and its descendants (LineageFS, Hi-Fi, LPM, SPADE, CamFlow) captured whole-system data lineage with acceptable overhead. None of it shipped. Today, provenance thrives only where it is expensive and mandatory — enterprise ML pipelines (MLflow, W&B, OpenLineage) and data catalogs — while the layer where 99% of the world’s files actually live (phones, laptops, desktops) records nothing. We argue this is a durable, load-bearing gap, not an abandoned backwater: the barriers that killed adoption (capture overhead, opaque binaries, cold-start history, privacy of the ledger itself, absent query UX) now have answers that did not exist in 2006 — eBPF/EndpointSecurity/ETW capture substrates, content-addressed compaction, local analytical engines (SQLite/DuckDB), on-device models that can read and summarize a file’s causal context, and C2PA, which is teaching the industry to demand provenance for media and has no answer for the other file types. We specify **provenance-native storage**: a causal event ledger per file system (inode-attributed records of create/transform/read lineages with process, binary, network-origin, and content-hash bindings), a query algebra over it (“where did this come from”, “what breaks if I delete this”, “reproduce this artifact”, “which files came from this site or app”, “what did this executable touch”), compaction and privacy semantics for the ledger itself, and the OS-integration surfaces that turn provenance from a forensics tool into a daily instrument. We design the system, quantify the barrier analysis, define the evaluation (overhead, lineage reconstruction accuracy, deletion-impact prediction), and confront the strongest objections: that provenance is a security-vendor feature, that nobody asks where files come from, that the ledger is a surveillance liability, and that vendors will never ship it.

Keywords: p-rick

p-rick working paper P-005 · series II (systems) · draft 1.0

Every file on a personal computer is an amnesiac. It knows its size and its modification time; it has forgotten who created it, from what inputs, through which transformations, over which network origin, and with which degree of trust. Twenty years ago the research community proved that operating systems *can* record this causal history automatically — the Provenance-Aware Storage System (PASS, FAST 2006) and its descendants (LineageFS, Hi-Fi, LPM, SPADE, CamFlow) captured whole-system data lineage with acceptable overhead. None of it shipped. Today, provenance thrives only where it is expensive and mandatory — enterprise ML pipelines (MLflow, W&B, OpenLineage) and data catalogs — while the layer where 99% of the world’s files actually live (phones, laptops, desktops) records nothing. We argue this is a durable, load-bearing gap, not an abandoned backwater: the barriers that killed adoption (capture overhead, opaque binaries, cold-start history, privacy of the ledger itself, absent query UX) now have answers that did not exist in 2006 — eBPF/EndpointSecurity/ETW capture substrates, content-addressed compaction, local analytical engines (SQLite/DuckDB), on-device models that can read and summarize a file’s causal context, and C2PA, which is teaching the industry to demand provenance for media and has no answer for the other file types. We specify **provenance-native storage**: a causal event ledger per file system (inode-attributed records of create/transform/read lineages with process, binary, network-origin, and content-hash bindings), a query algebra over it (“where did this come from”, “what breaks if I delete this”, “reproduce this artifact”, “which

files came from this site or app”, “what did this executable touch”), compaction and privacy semantics for the ledger itself, and the OS-integration surfaces that turn provenance from a forensics tool into a daily instrument. We design the system, quantify the barrier analysis, define the evaluation (overhead, lineage reconstruction accuracy, deletion-impact prediction), and confront the strongest objections: that provenance is a security-vendor feature, that nobody asks where files come from, that the ledger is a surveillance liability, and that vendors will never ship it.

1 Introduction

A file arrives on a laptop — say, `invoice_final_v3.pdf`. A human who wants to know its history can, with effort, reconstruct fragments: the download folder suggests an origin site (or an email client, or a messaging app); the creation time suggests when; Spotlight can find the filename; the PDF’s own metadata might carry a producer string. Everything causal — which process wrote it, from which parent files, via which network endpoint, on which device, after which chain of tools touched it — is gone the moment it would have been recorded. Multiply by the median household’s tens of thousands of files across five devices, and the aggregate picture is a civilization-scale archive with total amnesia about its own provenance.

The consequences are not abstract. Security answers (“did this document come from that phishing site?”) are unanswerable at the layer where users need them. Reproducibility of ordinary work (“rebuild this exact analysis from its actual inputs”) is reserved for scientists who discipline themselves with git and Docker, not for the spreadsheet that consumed four CSVs a colleague emailed. Deletion is blind: no system can say what breaks if a folder disappears, so people hoard everything. Trust in AI-generated artifacts is collapsing precisely because files carry no machine-checkable history of their making; C2PA content credentials patch this for *images and video at creation time* but say nothing about the PDF an AI assembled, the dataset an agent curated, or the file that traveled through three untrusted machines. And when a laptop dies or a phone is migrated, the causal layer — the soft knowledge of where things came from — dies with it, because it was never written down.

The research community solved the *capture* half of this problem a generation ago. PASS demonstrated kernel-level provenance capture with low single-digit overhead; its successors refined the model (whole-system graphs, filtered capture, hardware-assisted logging) and proved the security applications (intrusion detection, forensics, exfiltration analysis). What the community never produced — and what no product line has since fielded — is the *user-facing* system: provenance as a property of ordinary storage, queryable by ordinary questions, integrated into ordinary file management. This paper specifies that system and argues the gap between provenance research and provenance products is now the most valuable unfilled space in client-systems design.

2 The problem: causal amnesia at the storage layer

File systems record *attributes* (name, size, timestamps, permissions, maybe an extended attribute or two) and *not causation*. Four working gaps follow.

Origin amnesia. Windows keeps the Zone.Identifier alternate data stream (Mark of the Web, since Internet Explorer 5) for files downloaded via mainstream browsers, and macOS keeps a quarantine extended attribute — both are origin *flags*, not histories: one bit of “came from the internet,” no endpoint, no chain, no transform, nothing for files that arrived by sync, messaging, AirDrop, USB, or inside archives. The information needed to answer “where did this come from” existed at arrival time in the kernel, in the network stack, and in the writing process — and was discarded.

Process amnesia. A file was written by *some* process; no system records which binary, which parent process, which command line, or which *inputs that same process read in the same session*. This is precisely the data kernel-level provenance research showed is capturable. Its absence means attribution after the fact — “which app corrupted my documents folder?” — is a forensic investigation, not a query.

Reproduction amnesia. The transformation `out.xlsx = f(a.csv, b.csv)` leaves no trace of `f`, `a`, `b`, or their versions. Versioning systems (git, Dropbox history, macOS document versions) preserve *states* of files but never *the causal edges between them*. The result: work is reproducible only by people who impose external discipline (commit everything, containerize everything), which is to say, almost nobody.

Impact amnesia. No storage system can answer the deletion question — “what depends on this directory?” — because dependency edges are not recorded anywhere below the application layer. Humans respond rationally to blindness: they hoard. Storage bloat is partly a causality failure, not a psychology failure.

These four amnesias share one root cause: the layer that mediates every file operation (the kernel) has never been asked to persist the causal graph those operations imply, and the layer that owns file semantics (applications) has never had an interface to declare them. Neither alone can fix it — the kernel sees causality but not meaning; applications know meaning but not each other’s causality. The fix must be a *contract between the two*, which is the design of §5.

3 Twenty years of provenance research, zero products: the landscape

The academic lineage is deep and honest about what was achieved. **PASS** (Muniswamy-Reddy, Seltzer, et al., FAST 2006; “Provenance as data” follow-up) attached provenance records to inodes in a modified kernel, demonstrated W3C-PROV-adjacent lineage queries, and priced capture overhead in the low single digits — the founding result. **LineageFS** (IBM, 2009) exposed lineage as virtual shadow directories in the file namespace. **Hi-Fi** (2008) and **SPADE** (the DARPA Transparent Computing graph store) pushed whole-system fidelity and multi-host aggregation. **LPM** (Linux Provenance Modules, 2015) built the in-kernel framework; **CamFlow** (2017-18) delivered practical whole-system provenance on mainstream Linux via kernel patches, later eBPF-adjacent tooling, with flow-based semantics — the most field-ready research artifact in the line, deployed in research settings only. The security applications were proven repeatedly: provenance graphs ground intrusion-detection and exfil analysis (the ATT&CK-adjacent detection literature, e.g. ProvDetector-class systems and NDSS-line work on provenance-based detection).

Adjacent *product* lines cover the rest of the map, and none fills the center. **Enterprise data lineage** (OpenLineage + Marquez, DataHub, Amundsen, Snowflake/BigQuery lineage UIs) tracks pipelines between tables — provenance for data that lives in warehouses, reachable only through BI tooling, priced for enterprises. **ML experiment tracking** (MLflow, Weights & Biases, DVC) is the *voluntary* discipline layer for model artifacts — adoption driven by pain, confined to ML teams. **Reproducibility packagers** (CDE, ReproZip) capture execution environments on demand — the researcher’s one-shot camera, not a storage property. **Versioning** (git, git-annex, Datalad) records deliberate snapshots with human-authored causal messages; the causal edges between files (which inputs produced which outputs) exist only in commit-message prose. **Mark-of-the-Web** and quarantine xattrs: one-bit origin flags, browser-only, unchained. **Audit telemetry** (auditd, Windows ETW, macOS EndpointSecurity consumers, EDR stacks) records process–file events at enormous rate — but as *security telemetry* shipped to collectors and retained for days, not as *storage semantics* queryable for years alongside the files it describes. **C2PA/Content Credentials** (2021–, Adobe-led, now adopted by camera

makers and AI vendors) attaches signed provenance to media *at creation* — an origin certificate for pixels, inapplicable to the general file universe (a C2PA manifest cannot say “this PDF was derived from those three spreadsheets on this laptop”). **Semantic file research** (from Gifford’s 1991 semantic file systems to 2024-era LLM file organizers) attacked *retrieval* by content, never causality.

The center — automatic causal lineage for ordinary personal files, queryable by ordinary users, retained with the files themselves — remains empty in research *and* product. That a 2006 result with single-digit overhead produced no consumer follow-through in two decades demands explanation, not repetition. §4 supplies the barrier analysis; the barriers, we claim, have quietly expired.

4 Why it never shipped — and why the barriers expired

Barrier 1: capture required kernel surgery. PASS, Hi-Fi, LPM, CamFlow all patched or instrumented the kernel; vendors never accepted the patches, and out-of-tree modules are a non-starter for consumer machines. *Expired because:* every major OS now exposes a blessed kernel-event interface — eBPF on Linux (file and process events in-kernel, zero patches), Windows ETW kernel file/process providers, macOS EndpointSecurity (the API the entire EDR industry already runs on). Capture is now a *userspace* problem with vendor-supported primitives. This is the single largest change since 2006.

Barrier 2: overhead and storage blow-up. Whole-system provenance is a firehose; early systems burned disk proportional to I/O. *Expired because:* (a) the firehose is filterable — provenance need only record *persistent writes* and their input snapshots, a tiny fraction of I/O; (b) content-addressing deduplicates records (a hash appears once, records reference it); (c) NVMe and terabyte-scale personal storage make a 0.5–2% provenance budget acceptable where 2006 disks made 5% painful; (d) append-only ledgers with periodic compaction (collapse read events into summaries, roll process sessions into aggregates) bound the growth rate.

Barrier 3: opaque binaries — the semantics wall. Kernel capture sees `process P read a.csv, wrote out.xlsx` but cannot know P was a *merge* that semantically combined a and b, versus a *copy* versus a *transform*. The graph is causally true but semantically thin. *Partially expired:* this is where the AI era contributes — on-device models can classify transformations from observable traces (command line, I/O shape, file types) and, increasingly, applications can *declare* transforms through the API of §5. Semantics now improve with time instead of blocking the design: the ledger records facts, models annotate hypotheses, declarations upgrade hypotheses to facts.

Barrier 4: cold start. A provenance layer born in 2025 knows nothing about the 200,000 files already on the disk. *Expired because:* cold-start lineage can be *bootstrapped*: archive origin xattrs (MOTW, quarantine), package-manager manifests (which files belong to which app and version), sync-service histories (Dropbox/Drive/Photos timestamps and remote endpoints), and file-content identification (magic bytes, embedded metadata, format-specific creators) reconstruct an approximate causal past for legacy files. Imperfect lineage for old files, exact lineage for new ones — acceptable because the value accrues forward.

Barrier 5: the ledger is a surveillance liability. A causal history of who did what to which file is exactly what an attacker, an employer, or a subpoena wants. *Expired only by design:* the ledger must be (a) local-first, encrypted at rest, never leaving the device by default; (b) scoped — the *user* owns it, not the vendor; (c) redactable with integrity preserved (provenance from a Merkle-style chain, with deletion supported by re-anchoring, deliberately weaker than blockchain immortality — the design principle is *forgetting must be possible, so trust is calibrated not absolute*); (d) presented with a disclosure model borrowed from app permissions: an app

that declares transforms sees what it declared; the global graph is visible only to the user. We treat barrier 5 as the *live* barrier — §7 designs for it rather than around it.

Barrier 6: no query UX, so no demand, so no UX. *Expired by the smartphone:* users already ask provenance questions constantly — “where did this download go,” “what is this file,” “is this the version I edited” — they just ask them of Google and file managers that cannot answer. The demand exists latent; the query surfaces (finder/inspector panels, share-sheet “origin” cards, search filters like `from:app`, `week`, `site`) are design work, not research.

5 The design: a causal ledger with an application contract

5.1 The record space

The ledger is an append-only, content-addressed event log on the same volume (or a sibling volume) as the files it describes. Core record types:

- **Artifact** {content-hash, size, first-seen-t, mime} — every distinct content object, referenced by hash from all other records (dedup).
- **Write** {artifact, path, actor, parents[], t} — actor = process session (binary hash + version + invocation), parents = artifact hashes read in the same causal session (the *provenance edge set*).
- **Origin** {artifact, channel, endpoint-ref, t} — how content *entered* the device: network (endpoint, SNI/domain), messaging (app, conversation id), sync service (remote + remote-time), removable media, AirDrop-class local transfers.
- **Transform** {actor, declared-op, inputs→output, params} — application-declared semantics (see 5.2); records the human-meaningful operation, not just the edge.
- **Access** (aggregated) {actor, artifact, window} — reads, rolled up by compaction; retained at coarse granularity because reads are the firehose and the least valuable edges.
- **Epoch / Anchor** — periodic Merkle-style checkpoints enabling tamper-evidence *and* redaction (see 5.4).

Records reference artifacts by hash and paths as *secondary*, *mutable* labels — the graph is content-identity-based, so renames and moves are metadata, not identity events; identity survives path churn, which is what makes “where did this come from” answerable years later.

5.2 The application contract

The kernel-side capture is automatic and universal (via eBPF/ES/ETW): every persistent write gets a Write record with process-attributed parents. The contract adds semantics: an application that wants *understandable* provenance declares transforms via a small API — `declareTransform(op, inputArts, outputArt, params)` — at the natural moment (save, export, render, sync). Declared transforms render in the UI as meaningful nodes (“Exported to PDF from pages 1–3”) and confer benefits: declared-transform artifacts are exempt from ledger compaction, earn richer reproduction metadata (params recorded), and can carry user-facing trust badges. The split is deliberate: **the kernel guarantees facts, applications volunteer meanings, and models annotate the gap**. Unmodified apps still get full causal graphs (thin semantics); upgrading apps improves semantics without changing capture.

5.3 The query algebra

The ledger supports a small, composable query language over the typed causal graph, with five canonical query families as first-class citizens:

1. **Origin(art)** $\rightarrow$ the full ingress chain: endpoint, channel, time, and transit graph (which machines/apps handled it on this device).
2. **Lineage(art)** $\rightarrow$ DAG of ancestors/descendants with declared transforms as labeled edges (the “family tree” view).
3. **Impact(art | path)** $\rightarrow$ descendants that exist *because of* this artifact, joined against live path state — the deletion advisory (“3 documents cite this dataset; 1 app config reads it”).
4. **Reproduce(art)** $\rightarrow$ the executable recipe: input versions (from content-addressed snapshots where retained), transforms with params, and environment fingerprints — best-effort by construction (declarations exact, inferred edges probable).
5. **Provenance-Scan(filter)** $\rightarrow$ inverse queries: all artifacts from endpoint E / app A / site S / time-window W (the trust and hygiene queries: “show me everything that phishing domain touched”).

The algebra composes ($\text{Origin} \circ \text{Lineage}$), supports set filters, and joins against the live file index — deliberately a *local DuckDB/SQLite-shaped* workload, not a graph database: query volumes are human-scale, and the win is integration (the query engine lives where the files live).

5.4 Retention, compaction, and the right to forget

Ledger growth is bounded by compaction: Access records roll into windows; intermediate artifacts of declared transforms can be summarized into compound records (“session produced 47 intermediates, all derivable”); old snapshots expire by policy while hashes persist as identities. Redaction is a first-class operation with honest semantics: deleting a node re-anchors subsequent epochs (later verifiers can tell that *something* was redacted and when, not *what*). The threat model for tamper-evidence is the local attacker who wants to rewrite history invisibly; the design makes rewriting detectable without making forgetting impossible — a privacy-preserving compromise that pure blockchains get wrong for personal data and audit logs get wrong for users. Ledger default: local, encrypted, device-bound; export/share is explicit per artifact (“share this file’s provenance” as an attachment-like action — the C2PA-shaped export path for everything C2PA does not cover).

5.5 The surfaces

Provenance that lives only in a query CLI serves forensic engineers. The surfaces that make it a daily instrument: (a) **inspector panel** — origin card + lineage strip in the file manager’s info pane; (b) **download/share sheets** — origin card at the moment of arrival (the phishing check at the moment of doubt); (c) **delete confirmations** — impact summary instead of “are you sure”; (d) **search filters** — **from:** as a first-class predicate next to name/date/type; (e) **trust badges** on AI-assembly artifacts whose lineage shows declared transforms (the honest-AI hook). Each surface is a small consumer of the algebra; none requires new research.

6 Evaluation design

The reference implementation (Phase 1: Linux, eBPF capture, SQLite ledger, CLI + a GNOME-Files inspector prototype) is evaluated on four axes:

- **Overhead:** CPU and IO overhead of capture on interactive workloads (browsing, office, builds, model runs) targeting $< 2\%$ CPU and ledger growth $< 2\%$ of bytes-written-per-day with default compaction; capture overhead measured against the no-ledger baseline on identical activity.
- **Fidelity:** lineage reconstruction accuracy against ground truth on a scripted corpus (500 chained operations across 12 apps): edge precision/recall for kernel-only capture; semantic accuracy for declared-transform nodes; measured on the cold-start bootstrap for legacy files.
- **Utility:** the deletion-impact question in a user study ($n=20$): participants predict breakage of a move/delete with and without impact summaries; success rate and time-on-task.
- **Privacy audit:** adversarial redaction tests — verify the re-anchoring preserves epoch-integrity while removing content, and that export-by-default does not occur (network trace of the daemon must be silent).

7 Threats, objections, and honest answers

“This is an EDR feature; security vendors already ship it.” EDR telemetry is the *dual*: same events, inverted ownership (vendor/employer sees, user does not), inverted retention (days, not years), inverted interface (console alerts, not file-manager queries). The objection proves the capture substrate works and misses the product entirely: provenance-native storage is the user-owned inversion of EDR.

“Nobody asks where files come from.” Users ask constantly in degraded form (searching for downloads, hoarding “just in case,” forwarding documents they cannot vouch for). The absence of answers has trained the absence of questions; the smartphone-era UX patterns (origin cards, impact summaries) are the untraining. Honest concession: daily-query rate will start low; the killer applications are episodic (security doubt, migration, reproduction, audit) — a reference-checking instrument, not a feed.

“The ledger is a surveillance liability.” The live objection, designed for in §5.4: local-first, encrypted, redactable-with-integrity, permission-scoped. The concession is real: any causal record is coercible by sufficiently empowered adversaries; the design goal is that the ledger be *less* coercible than the artifacts it describes (the files themselves), while far more useful to their owner. Where that balance fails (regulated environments), the answer is policy: the ledger ships disabled where law forbids it.

“Vendors will never ship it; apps will never declare transforms.” The bootstrap avoids both: kernel-side capture works with zero app participation, surfaces work with zero vendor blessing on Linux, and the declaration API is an *upgrade path*, not a dependency. The beachhead sequence: enthusiasts (Linux tooling) → privacy-conscious platforms → one major app declaring transforms (a document editor is enough) → platform adoption when the trust surface (AI-era provenance badges) becomes a marketing feature. If no vendor ever ships it, the open-source path stands on its own — the same trajectory auditd/eBPF tooling took.

“Provenance will be wrong (inferred edges, cold start) and wrong provenance is worse than none.” Confusing recorded facts with inferred semantics: Write edges (kernel-recorded) are not wrong; transform labels can be, and are therefore *typed* as declared vs inferred, with confidence surfaced in the UI. The design principle — never present inference as fact — is exactly what the C2PA ecosystem learned; we inherit it.

8 The AI era makes this more necessary, not less

Three convergence forces make 2026 the moment this gap becomes unbearable. **Generated-content flood:** as AI produces a majority of new documents, images, and code, artifact trust collapses to provenance; C2PA covers pixels at creation, but the majority of value flows through *derived and assembled* artifacts (the report containing AI images, the dataset curated by an agent, the codebase patched by a model) whose provenance is causal-chain-shaped, not certificate-shaped. **Agent filesystems:** autonomous agents read, write, and transform personal files at machine speed; without a causal ledger, their effects are unauditable after the fact — with one, every agent action leaves edges a human can inspect (“the agent read these, wrote that, from which site’s data”). **On-device models** make the ledger *readable*: a causal graph is only useful if a human can absorb it; models that summarize lineages, translate query intent into algebra, and annotate transforms close the loop the 2006 systems left open. Provenance-native storage is, in this light, the trust infrastructure AI just made mandatory.

9 Roadmap

Phase 1 (this paper + reference implementation): Linux eBPF capture, ledger + compaction, query CLI, inspector prototype, evaluation per §6. Phase 2: macOS (EndpointSecurity) and Windows (ETW) agents; sync-service bootstrap; declaration API v1 with two open-source applications. Phase 3: provenance export format (C2PA-compatible extension for non-media artifacts); agent-action auditing mode; multi-device ledger federation (the mesh question: how ledgers merge when files move between devices — an open problem adjoining P-007). Open problems: inference-model training for transform classification; ledger compaction under adversarial pressure; the formal semantics of partial lineage under redaction; provenance-aware storage layout (co-locating ledger and volume for atomicity).

10 Conclusion

The file system is the last great amnesiac of personal computing. The research line that could have cured it proved capture feasible twenty years ago and then died in prototypes, for reasons — kernel surgery, disk prices, semantic opacity, cold start, surveillance fear, missing UX — that the intervening two decades have systematically dismantled. What remains is the work this paper specifies: an event ledger keyed by content identity, an application contract that upgrades facts into meanings, a query algebra answering the questions users already ask in degraded forms, retention semantics that treat forgetting as a feature, and surfaces that make causality a daily, legible property of storage. The files we will care about in five years — AI-assembled, agent-curated, machine-speed — will be trustworthy exactly to the degree their histories are. The storage layer should start remembering.

Disclosure

p-rick is an independent research program; no vendor has commissioned or reviewed this work; the author has no financial interest in cited systems. The companion paper P-007 addresses the device-mesh layer; P-005’s multi-device federation question (§9) deliberately defers to it.

References

1. Muniswamy-Reddy, K.-K., Holland, D. A., Braun, U., Seltzer, M. I., et al. (2006). *Provenance-Aware Storage Systems*. USENIX FAST.
2. Muniswamy-Reddy, K.-K., et al. (2009). *Provenance as data* / PASS follow-on lineage work. Harvard/HP.
3. Sar, C., Cai, M., Lin, L., et al. (2009). *LineageFS* (shadow-directory lineage file system). IBM Research.
4. King, S. T., & Cai, P. M. (2008). *Hi-Fi: collecting high-fidelity whole-system provenance*. UC Davis (SRI lineage line).
5. Bates, A., et al. (2015). *Linux Provenance Modules (LPM)*. ACM ASH.
6. Pasquier, T. M. F., et al. (2017–2018). *CamFlow: whole-system provenance / Runtime analysis of whole-system provenance*. IEEE CNS 2018 (the field-ready research artifact).
7. Simmhan, Y., Plale, B., & Gannon, D. (2005). *A survey of data provenance in e-science*. SIGMOD Record 34(3).
8. Braun, U., et al. *SPADE* / DARPA Transparent Computing provenance infrastructure.
9. ProvDetector-class and NDSS-line provenance-based intrusion detection (e.g., *ProvDetector*, 2020; *ShadeWatcher*, NDSS 2022).
10. CDE: Guo, P. J., & Engler, D. (2012). *Using machine learning to identify portable and non-portable C binaries*. USENIX ATC / ReproZip: Chirigati, F., et al. (2016). SIGMOD demo line.
11. Zaharia, M., et al. (2018). *MLflow*. / Lam, S. et al. W&B line; OpenLineage (2021–) specification.
12. W3C PROV Working Group (2013). *PROV-DM: The PROV Data Model*. W3C Recommendation.
13. Microsoft (2000–). *Mark of the Web / Zone.Identifier*; Apple quarantine extended attributes.
14. Adobe-led C2PA / Content Credentials coalition (2021–). *C2PA technical specification*.
15. Gifford, D. K., et al. (1991). *Semantic file systems*. ACM SOSP.
16. Gregg, B. (2019). *BPF Performance Tools* (file/process eBPF capture substrate). Addison-Wesley.
17. Apple Endpoint Security framework documentation (2019–); Windows Event Tracing (ETW) kernel file/process providers.
18. Stonebraker, M. lineage adjacency; Snowflake/BigQuery data-lineage product documentation (2023–).
19. Gemmell, J., et al. (2002). *MyLifeBits: fulfilling the Memex vision*. ACM Multimedia (personal capture antecedent).
20. Seltzer, M., et al. (2006–2011). The PASS research line, Harvard University (the founding demonstration).